CARBON-AWARE EV DEPOT ORCHESTRATOR

Problem • Proposed Solution • AI • AWS • MVP • Demo

Discussion document for team / mentor feedback

1. Executive Summary

Idea in one sentence: Build an AI-assisted operations system for an electric delivery depot that continuously decides when each EV should charge, balancing grid carbon intensity, electricity cost and depot constraints, while guaranteeing that every vehicle reaches its required state of charge before departure.

This is not primarily an EV charging app. It is a constrained scheduling and re-planning problem. The environmental objective is to shift flexible charging into cleaner hours, but the system must never sacrifice an operational requirement such as a vehicle leaving on time.

- Primary user: depot / fleet operations manager.
- Primary decision: which vehicle charges when, for how long, and at what power level.
- Primary impact metrics: kg CO_2e avoided, electricity cost saved, peak power reduced, and departure reliability.

2. Why This Fits the Hackathon

- Track: Waste and Energy — specifically EV energy use and carbon-aware electricity consumption.
- This is an open-innovation proposal inside the track, not an organizer-provided problem statement.
- The supplied organizer guidance emphasizes going beyond dashboards to show what a person should do differently because of the information.
- Asynchronous judging makes a visible decision loop especially valuable: schedule → disruption → agentic re-plan → validation → approval → result.
- The supplied prior-winner research identified carbon/grid-aware EV fleet charging as an open lane among the reviewed hackathon winners, while Amazon/AWS sustainability programs show interest in adjacent areas such as EV charging, demand balancing and energy scheduling.

3. The Problem, Properly Explained

3.1 The real-world scenario

Imagine an electric delivery depot with 40 vans and 10 chargers. Vans return at different times and with different battery levels. Each van has a different next departure time and a required state of charge (SOC). Meanwhile, electricity price and grid carbon intensity change during the day, and the depot has a finite electrical capacity.

- Charging everything immediately may create a large power peak.
- Charging during expensive tariff periods increases operating cost.
- Charging during carbon-intensive grid periods creates avoidable emissions.

- Charging too many vehicles simultaneously can exceed the depot transformer/site limit.
- Delaying a vehicle too much can cause it to miss its next departure.

The central question is: How can we find the cleanest and cheapest charging schedule that is still physically and operationally feasible?

3.2 Why carbon intensity matters

The same 20 kWh of electricity can have different associated emissions depending on when that electricity is consumed. A cleaner grid window can therefore reduce emissions without reducing the vehicle's required energy.

Illustrative example: 20 kWh × 0.70 kg CO_2e/kWh = 14 kg CO_2e, while 20 kWh × 0.30 kg CO_2e/kWh = 6 kg CO_2e. The energy is identical; the timing changes the estimated grid emissions.

These numbers are illustrative only. The actual prototype must use a declared carbon-intensity source or a clearly documented scenario model.

4. Example Depot

Vehicle	Current SOC	Required SOC	Departure	Illustrative energy need
V01	35%	90%	05:30	44 kWh
V02	55%	85%	06:30	24 kWh
V03	20%	95%	07:00	60 kWh
V04	70%	90%	08:00	16 kWh

The scheduler should not simply charge V01, V02, V03 and V04 in arrival order. It should consider deadline urgency, energy requirement, charger availability, grid carbon, tariff and site power.

5. What the Optimization Actually Does

5.1 Objective

Minimize $\alpha \cdot \text{Cost} + \beta \cdot CO_2e + \gamma \cdot \text{Peak/Infrastructure Penalty}$

- Cost = charging energy × time-dependent electricity tariff.
- CO_2e = charging energy × grid carbon intensity for that interval.
- Peak/infrastructure penalty discourages excessive simultaneous charging.
- α, β and γ can be configurable.

5.2 Hard constraints

- Every vehicle must reach required SOC before its departure deadline.
- A vehicle can charge only after it arrives.
- Charging power cannot exceed charger limits.
- Active charging sessions cannot exceed available chargers.
- Total charging power cannot exceed the depot transformer/site limit.
- Unavailable chargers cannot be assigned.
- If no feasible schedule exists, the system must say so rather than inventing one.

6. The Killer Feature: Re-planning When Reality Changes

The first schedule is not the main innovation. The system must keep working when reality changes.

1. At 18:00, the system creates a charging plan for 40 vehicles.
2. At 21:14, charger C04 fails.
3. The event is received by the system.
4. The agent retrieves current fleet, charger, tariff and carbon state.
5. The optimization tool generates a replacement schedule.
6. A deterministic validator checks every hard constraint.
7. The manager sees the new plan and its explanation.
8. The manager approves it.
9. The updated schedule is applied to the simulated charging layer.
10. The system continues monitoring and can re-plan again.

Core loop: Observe → Reason → Optimize → Validate → Human Approval → Execute → Monitor → Re-plan.

7. AI: What the Agent Does

7.1 Do not use an LLM for safety-critical optimization

The charging schedule should be produced by a deterministic optimization method such as CP-SAT / constraint programming or MILP-style optimization. The LLM should not be responsible for proving electrical feasibility or deadline satisfaction.

- Optimization tool: generates the schedule.
- Constraint validator: proves the schedule satisfies hard constraints.
- Agent: orchestrates tools, interprets events, explains recommendations and manages the approval workflow.

7.2 Strands / Bedrock agent role

- Get fleet state.
- Get charger and transformer state.
- Get grid-carbon forecast.
- Get tariff forecast.
- Call the optimization tool.
- Call the validation tool.
- Compare old and new schedules.
- Explain the reason for the change.
- Request human approval.
- Trigger execution after approval.

Example manager question: “Why did you delay Van 17 until 01:20?”

Example agent answer: “Van 17 departs at 06:30 and needs 24 kWh. I shifted charging to 01:20 because the forecasted grid carbon intensity and tariff are lower then. The optimizer verified that the required SOC will still be reached before departure and the transformer limit remains satisfied.”

Principle: the optimizer proves feasibility; the AI agent explains and orchestrates the decision.

8. AWS Architecture

AWS service	Purpose
Amazon Bedrock / Strands	Agentic orchestration, tool use, explanation and approval workflow.
AWS Step Functions	Plan → validate → approve → execute → monitor → re-plan state machine.
Amazon EventBridge	Triggers for grid/tariff updates and operational events.
AWS IoT Core	Vehicle/charger telemetry model for a realistic deployment architecture.
Amazon DynamoDB	Current vehicle state, charger state, plans, approvals and event state.
Amazon S3	Historical/simulation data, forecasts and artifacts.
AWS Lambda	Lightweight ingestion, APIs, validation and event handlers.
AWS Batch (optional)	Heavier optimization workloads if required.
Amazon CloudWatch	Logs, monitoring and operational metrics.
AWS Sustainability	Current AWS service for environmental-impact information about AWS usage; not the source of depot charging emissions.

2026 AWS note: The older Customer Carbon Footprint Tool was deprecated on June 30, 2026 in favor of AWS Sustainability. Do not present CCFT as the current standalone AWS product.

9. End-to-End Architecture

Grid / tariff / fleet event

↓

EventBridge

↓

Lambda / ingestion

↓

DynamoDB: current state

↓

Strands agent

↓

Fleet + grid + tariff tools

↓

Optimization engine

↓

Deterministic constraint validator

↓

Human approval

↓

Approved charging plan

↓

IoT / simulated charger control

↓

CloudWatch monitoring

↓

New event? → Re-plan

10. Data We Need

Data	Example fields	MVP approach	Why
Fleet	vehicle ID, SOC, capacity, arrival, departure, required SOC	Synthetic realistic data	Vehicle constraints
Chargers	charger ID, max kW, availability	Simulated telemetry	Charging constraints
Depot	transformer/site limit	Scenario configuration	Hard power limit
Tariff	₹/kWh by time	Declared tariff scenario	Cost objective
Grid carbon	gCO_2e/kWh by time	Public source where available; otherwise declared model	Carbon objective
Events	charger failure, late arrival, SOC deviation	Injected in demo	Re-planning

Important: Do not claim real-time nationwide Indian grid carbon intensity if the selected source does not provide that coverage. State the exact source, geography, resolution and limitations.

11. Carbon Accounting

Estimated charging CO_2e = Energy charged (kWh) × Grid carbon intensity (kg CO_2e/kWh)

The comparison should be made at the chosen time resolution rather than using a single annual average, because the project's value comes from shifting flexible charging between different grid conditions.

12. What We Measure

Metric	Meaning
CO_2e avoided	Baseline charging emissions minus optimized charging emissions.
Cost saved	Baseline electricity cost minus optimized electricity cost.
Peak power	Maximum simultaneous charging load before vs after optimization.
Energy shifted	kWh moved away from less desirable windows into feasible better windows.
Departure reliability	Vehicles meeting required SOC by deadline; target is 100% in

	feasible scenarios.
Re-plan latency	Time from disruption event to validated replacement schedule.

13. What We Should Build in the Hackathon

Keep the MVP narrow. A complete working loop is more valuable than ten unfinished features.

11. One simulated depot: 40 EVs, 10 chargers, one transformer/site power limit.
12. 24-hour tariff and grid-carbon timeline.
13. Vehicle arrival, SOC and departure data.
14. Optimization engine producing a schedule.
15. Dashboard showing schedule, cost, carbon and peak power.
16. One deliberate disruption: charger failure or grid-carbon change.
17. Strands agent detects the event and calls the optimizer.
18. Deterministic validator checks all constraints.
19. Human approval step.
20. Updated plan and impact numbers.
21. Public AWS deployment, repository, README and reproducible scenario.

14. Suggested 3-Minute Video

Time	What to show
0:00–0:25	40 vans return; explain the problem and why immediate charging is not always best.
0:25–0:55	Show fleet state, carbon/tariff timeline and initial schedule.
0:55–1:25	Show optimized schedule, cost, CO_2e and peak power; all departure constraints green.
1:25–2:05	Inject charger failure / fleet deviation and show the original plan becoming invalid/suboptimal.
2:05–2:35	Show Strands agent calling tools, generating a replacement plan and requesting approval.
2:35–2:55	Show new schedule, 0 missed departures and updated impact metrics.
2:55–3:00	Show AWS architecture and one-line impact statement.

15. Why This Can Be Competitive

- One named user and one operational decision.
- Environmental impact is quantified.
- The demo visibly changes when the world changes.
- Hard operational constraints make the engineering credible.
- AI has a legitimate role without being responsible for safety-critical math.
- AWS can be load-bearing across events, state, orchestration, agent tools, telemetry and observability.
- The concept has a natural scale story: 40 simulated vehicles → many vehicles → many depots.
- The supplied winner research shows AWS judges have rewarded agentic workflows, human approval, real data, measurable numbers and operational relevance.

16. Main Risks and How We Address Them

Risk / objection	Response
"This is just EV optimization."	Make the agentic operational loop the centerpiece: detect disruption → re-plan → validate → approve → execute.
"Where is the environmental impact?"	Put kg CO_2e avoided on the first screen and explain the hourly carbon accounting.
"Is AWS just hosting it?"	Use AWS for orchestration, state, event handling, agent tooling, telemetry and observability.
"Can you prove the plan is safe?"	Use a deterministic optimizer and explicit constraint validator; never rely on LLM output for feasibility.
"Your India carbon data is incomplete."	State the exact source and gap; use a declared scenario model where necessary.
"Are these Amazon fleet numbers?"	No. Clearly label the depot as a realistic simulation/prototype, not Amazon internal data.
"What if no feasible schedule exists?"	Return an explicit infeasibility explanation and identify the limiting constraint.

17. What We Should NOT Build / Claim

- Not a generic carbon dashboard.
- Not an EV charging locator.
- Not an LLM that directly decides charging schedules without hard validation.
- Not a claim that we control Amazon's real fleet.
- Not a claim that simulated savings are real-world savings.
- Not a nationwide real-time Indian grid-carbon claim unless the data supports it.
- Do not add batch compute scheduling to the MVP; it is better presented as a future extension.

18. Questions to Send to Reviewers

22. Would you immediately understand the problem and why it matters?
23. Does it feel environmental enough for the Waste and Energy track?
24. Is the depot operations manager a strong user persona?
25. Is the charger-failure/re-planning scenario compelling enough for a 3-minute asynchronous demo?
26. Does the AWS architecture feel genuinely load-bearing?
27. Would you trust the carbon-savings number if the source and assumptions are published?
28. Should we focus only on EV depot charging rather than adding compute?
29. What is the biggest reason you would reject this idea?
30. Would you choose this over a strong Air or Heat & Water project? Why?

19. Current Recommendation

Recommended framing: "An agentic operating system for electric delivery depots that continuously finds the lowest-carbon feasible charging plan while guaranteeing that every vehicle is ready for its next shift."

The key demo sentence: "We don't optimize for the greenest schedule. We optimize for the greenest schedule that the depot can actually operate."

20. References / Source Notes

- Team-supplied hackathon research: prior winners, judging patterns, track descriptions and open-lane analysis.
- AWS Sustainability Console (2026): https://aws.amazon.com/about-aws/whats-new/2026/03/aws-launches-sustainability-console/
- AWS Customer Carbon Footprint Tool release notes (including 2026 deprecation): https://docs.aws.amazon.com/ccft/latest/releasenotes/what-is-ccftrn.html
- AWS Step Functions + Bedrock AgentCore integration: https://docs.aws.amazon.com/step-functions/latest/dg/connect-bedrockagentcore.html
- AWS location-based emissions methodology context: https://aws.amazon.com/about-aws/whats-new/2025/06/customer-carbon-footprint-tool-location-based-emissions/

Important note: All fleet sizes, prices, carbon intensities and savings examples in this document are illustrative unless explicitly identified as sourced data. They should be replaced by the final prototype's documented assumptions and data sources.